

Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías
Análisis y Diseño de Algoritmos — 2026-1

Proyecto K-QGMIP

Manual Técnico

Partición de Mínima Información en k -particiones
Estrategias **KGeoMIP** (geométrica) y **KQNodes** (submodular)

Teoría de la Información Integrada (IIT)
 $k \in \{2, 3, 4, 5\}$ — n hasta ≈ 25 nodos

8 de junio de 2026

Índice

1. Resumen ejecutivo	3
1.1. Problema abordado	3
1.2. Enfoque algorítmico	3
1.3. Principales resultados	3
1.4. Limitaciones y recomendaciones de uso	3
2. Fundamentos teóricos	4
2.1. Sistema, subsistema y repertorio	4
2.2. Definición formal de k -partición	4
2.3. Función objetivo: la pérdida δ_k	4
2.4. Extensión del marco GeoMIP/QNodes a k -particiones	5
2.5. Complejidad del espacio de soluciones	5
3. Arquitectura del software	5
3.1. Diagrama de clases	6
3.2. Diagrama de paquetes	6
3.3. Diagrama de secuencia	6
3.4. Patrones de diseño aplicados	7
3.5. Decisiones arquitectónicas clave	7
4. Diseño algorítmico	8
4.1. Visión general	8
4.2. KGeoMIP: generación geométrica del <i>cut pool</i>	8
4.3. KQNodes: generación submodular del <i>cut pool</i>	9
4.4. Estructuras de datos	9
4.5. Estrategia de búsqueda	9
4.6. Evaluación de particiones	9
4.7. Optimizaciones implementadas	10
4.8. Metaheurísticas comparativas (GA / SA / Tabú)	10
5. Análisis de complejidad	10
6. Detalles de implementación	11
6.1. Métodos principales	11
6.2. Dependencias externas	12
6.3. Ingeniería de software	12
6.4. Tests implementados	12
7. Resultados experimentales	12
7.1. Datasets	12
7.2. Métricas	13
7.3. Tabla de resultados	13
7.4. Gráficas y visualizaciones	14
7.5. Visualización interactiva e interfaz web	15
7.6. Análisis de resultados	15
7.7. Validación de correctitud	16

8. Limitaciones y trabajo futuro	16
8.1. Limitaciones conocidas	16
8.2. Supuestos y restricciones	16
8.3. Mejoras potenciales y trabajo futuro	16
9. Apéndices técnicos	16
9.1. Reducción $k = 2 \Rightarrow$ bipartición legada	16
9.2. Reproducción de figuras y diagramas	17
9.3. Referencias	17
9.4. Uso de inteligencia artificial generativa	17

1. Resumen ejecutivo

1.1. Problema abordado

La **Teoría de la Información Integrada** (IIT) cuantifica cuánta información genera un sistema *como un todo* por encima de la suma de sus partes. El indicador central es Φ , que mide la pérdida de información al *cortar* el sistema según su **Partición de Mínima Información** (MIP): la división que menos información destruye. El sistema se describe mediante una **Matriz de Probabilidad de Transición** (TPM) sobre variables binarias.

El marco de referencia (GeoMIP y QNodes) resuelve únicamente **biparticiones** ($k = 2$). Este proyecto lo **extiende a k -particiones** con $k \in \{2, 3, 4, 5\}$ y escala el tamaño del sistema hasta $n \approx 25$ nodos. Esta generalización es relevante porque la verdadera estructura de un sistema rara vez es binaria: una partición en k bloques captura modularidad de grano más fino que un único corte.

1.2. Enfoque algorítmico

Se implementan dos estrategias núcleo, ambas heredan de la clase abstracta SIA y se evalúan con la *misma* pérdida δ_k :

- **KGeoMIP** (geométrica): interpreta el espacio de estados como un hipercubo, construye *una sola vez* la tabla de costos de transición T y aplica refinamiento jerárquico voraz ($k - 1$ biparticiones sucesivas).
- **KQNodes** (submodular): obtiene candidatos mediante una búsqueda tipo Queyranne y los refina con el mismo motor voraz que KGeoMIP.

Como apoyo se incluyen un algoritmo **exacto** (enumeración de Stirling, *ground truth* para n pequeño) y un **baseline** de *clustering* espectral/KMeans determinista.

1.3. Principales resultados

- **Correctitud:** para $k = 2$, KGeoMIP y KQNodes reproducen *exactamente* GeoMIP y QNodes (invariante de regresión). KGeoMIP coincide con el óptimo exacto en los casos golden y KQNodes acierta 9/10 frente al oráculo, corrigiendo además un defecto histórico de QNodes.
- **Calidad:** las dos estrategias núcleo obtienen pérdidas δ_k *órdenes de magnitud menores* que el baseline de clustering (p. ej. 0,47 vs. 4,3 en N10A, $k = 2$).
- **Eficiencia:** vectorización NumPy ($6\text{--}9\times$ en la ruta caliente), tensores float32 (el sistema $n = 25$ cabe en RAM) y paralelismo por procesos (PCD) con *speedup* real medido.

1.4. Limitaciones y recomendaciones de uso

KGeoMIP y KQNodes tienen techo práctico en $n \approx 15\text{--}20$ por la marginalización $\mathcal{O}(2^n)$ y la búsqueda $\mathcal{O}(n^3)$. Para $n = 25$ solo el baseline de clustering *propone* una partición en tiempo razonable, sin puntuarla con δ_k (requiere el subsistema completo en memoria). Se recomienda: KGeoMIP para análisis exactos/rápidos hasta $n \approx 15$; KQNodes cuando se busca robustez submodular; clustering solo como referencia exploratoria a gran escala.

2. Fundamentos teóricos

2.1. Sistema, subsistema y repertorio

Un sistema de n variables binarias se describe por una TPM determinista $T \in \{0,1\}^{2^n \times n}$ en orden *little-endian* (fila 0 = 00...0). Internamente cada columna se representa como un NCube: un tensor de forma $(2, \dots, 2)$ (n ejes). El subsistema bajo análisis se obtiene fijando condiciones de fondo (*condition*) y restando *purview/mechanism* (*subtract*). Su distribución marginal de efecto P es el repertorio que se desea preservar.

2.2. Definición formal de k -partición

Sea el subsistema con un *universo futuro* (*purview*) F de índices de efecto en $t+1$ y un *universo presente* (*mechanism*) M de índices de mecanismo en t . Una **k -partición** es una colección de k bloques

$$\mathcal{P} = \{ (F_1, M_1), (F_2, M_2), \dots, (F_k, M_k) \}$$

tal que las partes *futuras* forman una partición de F y las partes *presentes* una partición de M :

$$\bigcup_{j=1}^k F_j = F, \quad F_i \cap F_j = \emptyset \ (i \neq j); \quad \bigcup_{j=1}^k M_j = M, \quad M_i \cap M_j = \emptyset \ (i \neq j).$$

Semántica estricta. Se adopta la definición **estricta** (doc. oficial §2.1): los k bloques deben ser *no vacuos* (cada bloque tiene al menos un átomo, futuro o presente). Un *átomo* es un par (tiempo, índice) con tiempo $\in \{\text{ACTUAL}, \text{EFECTO}\}$. La validación se impone en `KPartition.__post_init__`: si algún bloque queda totalmente vacío se lanza `ValueError`. Esto evita la degeneración en la que $k > 2$ colapsa a una bipartición (un bloque vacío sería “inerte”), garantizando que la rejilla experimental sea genuina y la pérdida *monótona* en k .

Ejemplo ($n = 3$). Con $F = M = \{0, 1, 2\}$, una 3-partición válida es

$$\mathcal{P} = \{(\{0\}, \{0\}), (\{1\}, \{1, 2\}), (\{2\}, \emptyset)\}.$$

El bloque 3 es no vacuo (tiene el átomo de efecto 2) aunque su parte presente sea vacía; los tres bloques cubren F y M de forma disjunta.

2.3. Función objetivo: la pérdida δ_k

Cada k -partición induce una *reconstrucción factorizada* del subsistema: los bloques se evalúan de forma independiente y se recombinan por producto tensorial. La pérdida de información es la distancia *Earth Mover's Distance* (EMD) entre el repertorio original P y el reconstruido $P_{\mathcal{P}}$:

$$\delta_k(\mathcal{P}) = \text{EMD}(P, P_{\mathcal{P}}). \quad (1)$$

Bajo independencia condicional del repertorio de efecto, la EMD se reduce a la suma de diferencias absolutas componente a componente (EMD-efecto *analítica*, coste $\mathcal{O}(2^{|F|})$ sin matriz de costos):

$$\text{EMD}(u, v) = \sum_i |u_i - v_i|. \quad (2)$$

El problema de optimización es entonces

$$\mathcal{P}^* = \arg \min_{\mathcal{P} \in \Pi_k(F, M)} \delta_k(\mathcal{P}), \quad (3)$$

donde $\Pi_k(F, M)$ es el conjunto de k -particiones estrictas. La k -MIP es $\mathcal{P}^*$ y su valor $\delta_k(\mathcal{P}^*)$ generaliza a Φ .

2.4. Extensión del marco GeoMIP/QNodes a k -particiones

La clave de la extensión (doc. §2.2) es que **una k -partición equivale a $k - 1$ biparticiones sucesivas**: dividir el hipercubo con $k - 1$ hiperplanos es lo mismo que cortar un bloque en dos, repetidamente. Esto permite *reutilizar* la maquinaria de bipartición de ambas estrategias:

- GeoMIP propone cortes geométricos desde la tabla T ; KGeoMIP los proyecta sobre subbloques y elige, en cada paso, el corte que minimiza (1).
- QNodes propone cortes submodulares (Queyranne); KQNodes los refina con el mismo motor voraz.

Para $k = 2$ el refinamiento ejecuta *un* corte y por tanto recupera la bipartición legada exactamente (invariante 1). La justificación de calidad es doble: (i) cada corte hereda la garantía de su método base (GeoMIP es óptimo exacto en biparticiones sobre el núcleo verificado; Queyranne es exacto para funciones submodulares simétricas en $\mathcal{O}(n^3)$), y (ii) la elección voraz guiada por δ_k asegura *monotonía* y validación contra el exacto en n pequeño. El óptimo global de k -particiones *no* se garantiza (el voraz es heurístico), lo que se documenta honestamente en §8.

2.5. Complejidad del espacio de soluciones

El número de particiones de un conjunto de a átomos en exactamente k bloques no vacíos es el **número de Stirling de segunda especie** $S(a, k)$. Para el subsistema completo $a = |F| + |M| = 2n$, de modo que el espacio crece como

$$|\Pi_k| = S(2n, k) \sim \frac{k^{2n}}{k!} \quad (n \text{ grande}).$$

Comparado con biparticiones, donde $S(2n, 2) = 2^{2n-1} - 1$ es ya exponencial, el caso k -partita es *super-exponencial* en k . La enumeración exacta (Stirling) solo es viable para $n \lesssim 6$; de ahí la necesidad de las heurísticas KGeoMIP/KQNodes, que recorren un subconjunto dirigido de tamaño polinómico de candidatos.

3. Arquitectura del software

El sistema sigue el patrón **Strategy + Template Method** sobre la clase abstracta SIA. La Figura 1 muestra los componentes principales y su integración.

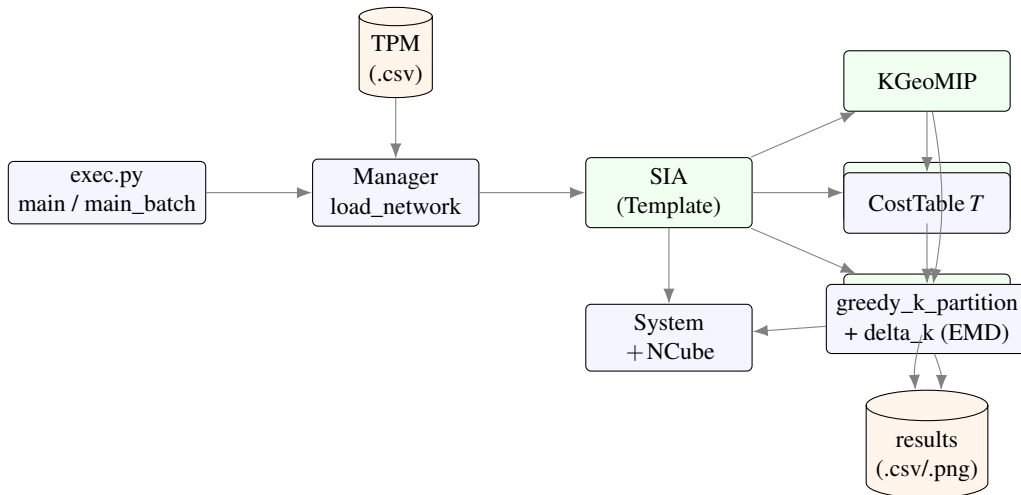


Figura 1: Arquitectura general: el Manager carga la TPM, cada estrategia hereda de SIA, comparte el motor greedy_k_partition y puntúa con delta_k.

3.1. Diagrama de clases

La Figura 2 resume la jerarquía de herencia y las relaciones de composición. La versión editable (PlantUML) está en docs/manuales/diagrams/02_clases.puml.

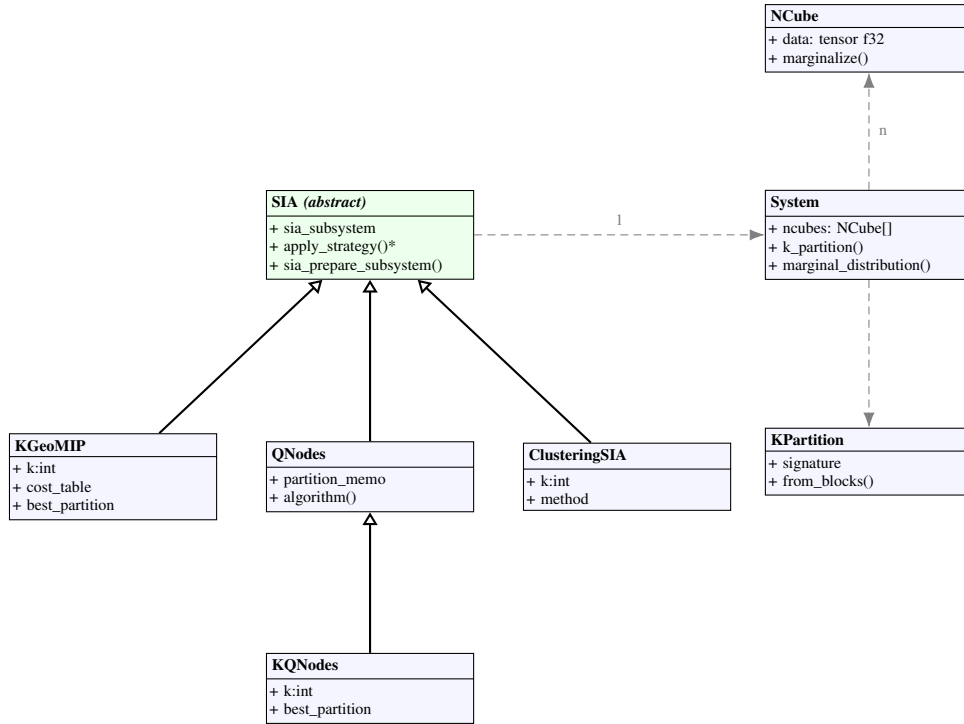


Figura 2: Diagrama de clases (núcleo). Flechas huecas: herencia; discontinuas: composición/dependencia. KGeoMIP y KQNodes producen una KPartition puntuada vía `System.k_partition`.

3.2. Diagrama de paquetes

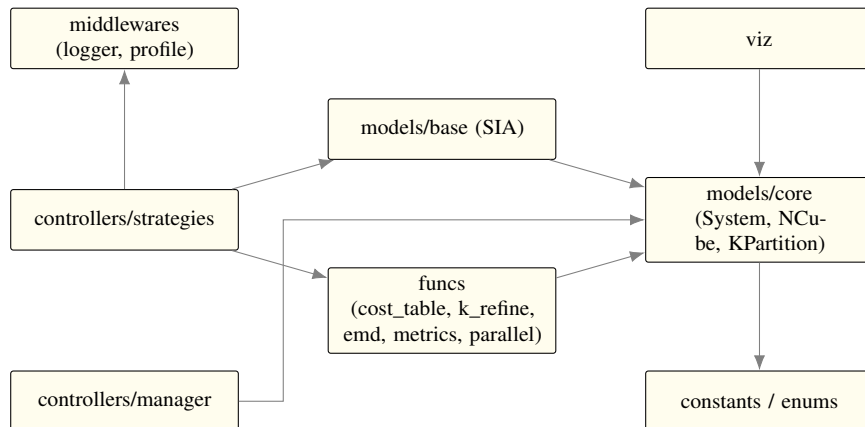


Figura 3: Organización modular en `src/`. Las estrategias dependen del modelo base, del núcleo de dominio y de `funcs`.

3.3. Diagrama de secuencia

La Figura 4 traza la búsqueda de la k -MIP con KGeoMIP.

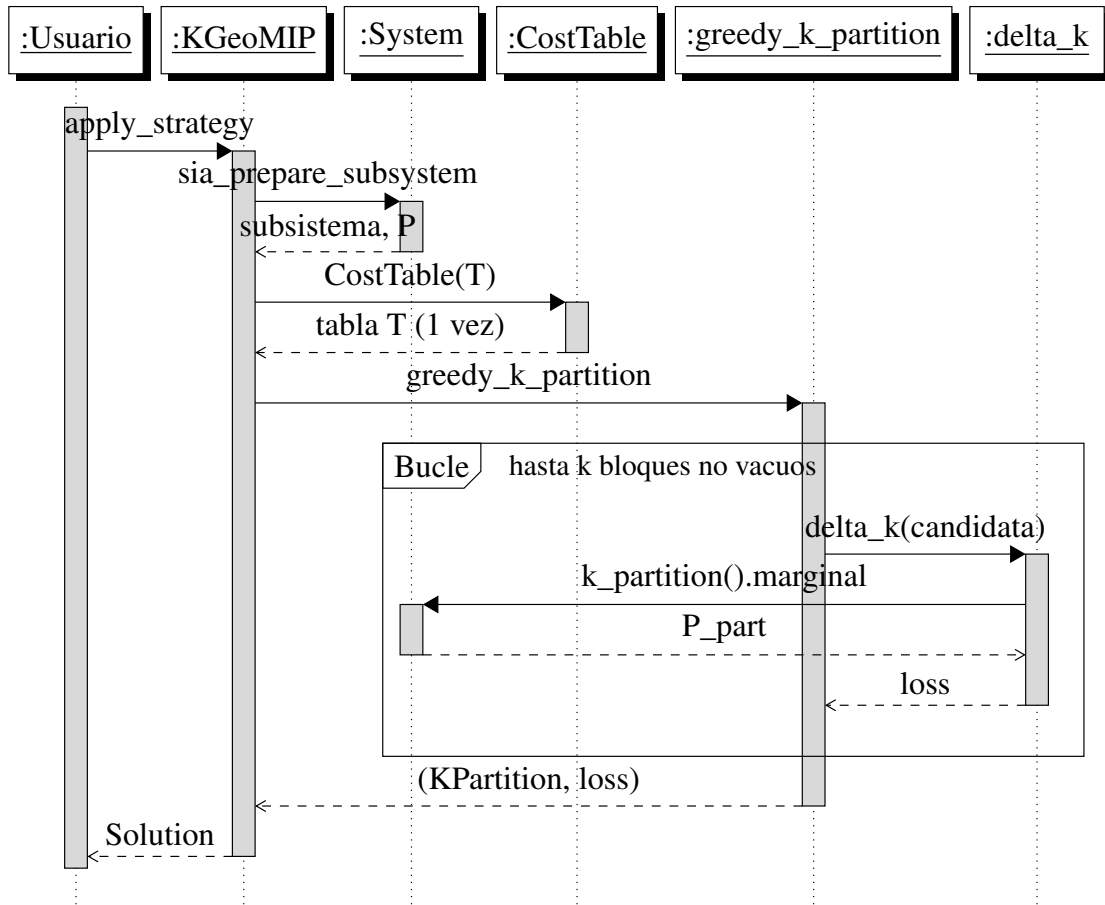


Figura 4: Secuencia de la búsqueda k -partita (KGeoMIP). La tabla T se construye una sola vez; el bucle voraz puntúa cada candidata con δ_k .

3.4. Patrones de diseño aplicados

- **Strategy:** cada algoritmo (KGeoMIP, KQNodes, Clustering, ExhaustiveK) es una estrategia intercambiable con interfaz común `apply_strategy`.
- **Template Method:** `SIA.sia_prepare_subsystem` fija el esqueleto compartido (`System` → `condition` → `subtract` → `marginal`); las subclases solo implementan la búsqueda.
- **Singleton:** `application` centraliza la configuración global (semilla, página de red, métrica, notación, EMD, profiling).
- **Objeto inmutable / value object:** `KPartition` se autovalida en construcción (cobertura, disyunción, k bloques no vacuos).

3.5. Decisiones arquitectónicas clave

- **Reutilización del corte base:** en lugar de reimplementar GeoMIP y QNodes, sus pools de candidatos se *proyectan* sobre subbloques. Garantiza el invariante $k = 2$ y reduce la deuda técnica.
- **Motor voraz compartido** (`greedy_k_partition`): un único punto de verdad para la construcción k -partita; ambas estrategias difieren solo en *cómo* generan el *cut pool*.

- **Tabla T una sola vez:** la estructura cara se calcula una vez por sistema y se reutiliza para todo k (doc. §3).
- **float32:** se sacrifica precisión irrelevante (validada a 10^{-5}) por la mitad de memoria, lo que habilita $n = 25$ en RAM.

4. Diseño algorítmico

4.1. Visión general

Ambas estrategias comparten la filosofía “*divide y refina*”: parten del subsistema como un único bloque y lo dividen $k - 1$ veces, eligiendo cada vez el corte que minimiza δ_k . Solo difieren en el origen de los cortes candidatos (geométrico vs. submodular). El Algoritmo 1 es el motor común.

Algoritmo 1 GREEDYKPARTITION — motor voraz compartido

Require: subsistema S , repertorio base P , *cut pool* C , universos F, M , entero k

Ensure: k -partición $\mathcal{P}$ con su pérdida δ_k

```

1:  $blocks \leftarrow [(F, M)]$  ▷ un solo bloque inicial
2: while  $|blocks| < k$  do
3:    $best \leftarrow \infty$ ;  $bestBlocks \leftarrow \text{NONE}$ 
4:   for cada bloque  $b = (F_b, M_b)$  en  $blocks$  con  $|F_b| + |M_b| \geq 2$  do
5:     for cada corte  $c = (F_c, M_c)$  en  $C$  do
6:        $in \leftarrow (F_b \cap F_c, M_b \cap M_c)$ ;  $out \leftarrow (F_b \setminus F_c, M_b \setminus M_c)$ 
7:       if  $in$  vacuo o  $out$  vacuo then continue
8:       end if
9:        $\mathcal{P}' \leftarrow$  reemplazar  $b$  por  $\{in, out\}$  en  $blocks$ 
10:       $\ell \leftarrow \delta_k(S, \mathcal{P}', P)$  ▷ Ec. (1)
11:      if  $\ell < best$  then  $best \leftarrow \ell$ ;  $bestBlocks \leftarrow \mathcal{P}'$ 
12:      end if
13:    end for
14:  end for
15:  if  $bestBlocks = \text{NONE}$  then error “no refinable a  $k$  bloques”
16:  end if
17:   $blocks \leftarrow bestBlocks$ 
18: end while
19: return (KPARTITION( $blocks$ ),  $\delta_k(S, blocks, P)$ )
```

4.2. KGeoMIP: generación geométrica del *cut pool*

KGeoMIP construye la tabla T (Algoritmo 2) y deriva de ella los cortes candidatos. T recorre el hipercubo por niveles de Hamming desde el estado inicial hacia su antípoda, almacenando un vector de costo por nodo, ponderado por $\gamma = 2^{-d_H}$.

Algoritmo 2 COSTTABLE — tabla de costos de transición T (una vez)**Require:** matriz plana por nodo X , estado inicial s_0 , antípoda $s_1 = 1 - s_0$

```

1:  $paths[0] \leftarrow \{s_0\}$ 
2: for  $level = 1 \dots n$  do
3:   for cada  $s$  en  $paths[level - 1]$  do
4:     for cada bit  $i$  con  $s_i \neq (s_1)_i$  do
5:        $s' \leftarrow s$  con el bit  $i$  volteado hacia  $s_1$ 
6:        $d_H \leftarrow \text{HAMMING}(s_0, s'); \quad \gamma \leftarrow 2^{-d_H}$ 
7:        $cost \leftarrow |X[:, \text{int}(s_0)] - X[:, \text{int}(s')]|$  ▷ gather vectorizado
8:       if  $d_H > 1$  then acumular costos de vecinos de un bit
9:       end if
10:       $T[(s_0, s')] \leftarrow \gamma \cdot cost$ 
11:    end for
12:  end for
13: end for
14: return  $T$ , candidatos =  $n$  cortes de nodo único + mejor corte por nivel

```

4.3. KQNodes: generación submodular del *cut pool*

KQNodes ejecuta una vez la búsqueda tipo Queyranne heredada de QNodes para poblar `partition_memo`. Cada clave del memo representa un lado de una bipartición candidata como colección de vértices (tiempo, índice); se aplanan y se separa por capa en un corte (F_c, M_c) . Para $k = 2$, el refinamiento aplica un único corte y reproduce QNodes exactamente (incluidos sus casos subóptimos conocidos, congelados en los tests de triaje).

4.4. Estructuras de datos

- **NCube** — tensor $(2, \dots, 2)$ float32 por columna de la TPM; `marginalize` es puro (devuelve un nuevo NCube) y memoizado.
- **System** — colección de NCubes con `condition`, `subtract`, `bipartition` (memoizada), `k_partition` y `marginal_distribution`.
- **CostTable** — matriz contigua $(n, 2^m)$ para el *gather* vectorizado de columnas; `transition_table` mapea $(s_0, s') \rightarrow$ vector de costo.
- **KPartition** — bloques pareados (F_j, M_j) como tuplas ordenadas; `signature` es la representación canónica usada para puntuar y comparar.
- **Block / cut** — par `frozenset` (F, M) usado en el motor voraz; dividir b con c da $(b \cap c, b \setminus c)$.

4.5. Estrategia de búsqueda

La técnica de diseño es **voraz jerárquica** (greedy): en lugar de enumerar $S(2n, k)$ particiones, se realizan $k - 1$ decisiones locales óptimas. El *cut pool* acota el ramaje a $\mathcal{O}(n)$ cortes por nivel. Se trata de una heurística: garantiza factibilidad y monotonía pero no optimalidad global; por eso se valida contra el exacto (§7).

4.6. Evaluación de particiones

Una candidata se puntúa con `delta_k` (Ec. (1)): se construye el subsistema particionado `System.k_partition( $\mathcal{P}$ )`, se obtiene su distribución marginal y se mide la EMD-efecto analítica (Ec. (2)) contra el repertorio base P .

4.7. Optimizaciones implementadas

- **Tabla T una vez** y reutilizada para todo k .
- **Vectorización NumPy:** *gather* de columnas y acumulación vectorial en `CostTable` (eliminado el formateo de cadenas por estado); conjuntos de Python en la ruta caliente de `bipartition`.
- **Memoización** de `bipartition/marginalize`.
- `float32` en los tensores (mitad de memoria).
- **Paralelismo por procesos** (PCD): `joblib/loky` con `SharedMemory`, semillas por proceso (`SeedSequence`) y control de afinidad de hilos (`OMP_NUM_THREADS=1`) para evitar sobre-suscripción.
- **Numba (opcional):** `batch_effect_emd` con umbral; el perfilado mostró que la EMD es $< 1\%$ del costo (dominado por la marginalización NumPy), por lo que Numba se reserva para lotes grandes.

4.8. Metaheurísticas comparativas (GA / SA / Tabú)

Como variante de comparación se implementan tres metaheurísticas (`src/funcs/metaheuristic.py`) que heredan de SIA y se puntúan con la *misma* pérdida δ_k , de modo que sus resultados son directamente comparables con KGeoMIP/KQNodes y con el óptimo exacto.

Codificación y operadores comunes. Una candidata es un *vector de etiquetas* $\ell \in \{0, \dots, k-1\}^{|F|+|M|}$ que asigna cada átomo (tiempo, índice) a un bloque. La decodificación a `KPartition` agrupa los átomos por etiqueta; si algún bloque queda vacío la candidata es *infactible*. La **reparación** restaura la sobreyectividad sobre los k bloques moviendo átomos desde bloques excedentes, garantizando una k -partición estricta (todos los k bloques no vacíos, §2). El evaluador memoiza $\delta_k(\ell)$ por vector canónico para no recomputar la marginalización.

- **Algoritmo genético** (`genetic_search`). Población de soluciones reparadas; selección por torneo, cruce uniforme por átomo, mutación por reetiquetado con probabilidad p_m y *elitismo* del mejor individuo.
- **Recocido simulado** (`simulated_annealing_search`). Toma δ_k como energía; en cada paso reetiqueta un átomo y acepta el vecino con probabilidad de Boltzmann $\exp(-\Delta\delta_k/T)$, con enfriamiento geométrico $T \leftarrow \alpha T$.
- **Búsqueda tabú** (`tabu_search`). Explora un vecindario de reetiquetados, escoge el mejor vecino *no tabú* (con criterio de aspiración si mejora el incumbente) y mantiene una lista tabú de tenencia fija.

Todas son **deterministas** bajo la semilla global (`application.numpy_seed`) vía `np.random.default_rng`. Por construcción nunca superan al óptimo exacto ($\delta_k \geq \delta_k^*$); los tests verifican que alcanzan δ_k^* en n pequeño (N3A/N4A) y el análisis empírico (§7) discute su comportamiento al crecer k .

5. Análisis de complejidad

Sea n el número de nodos, m las dimensiones de mecanismo del subsistema y k el número de bloques. Las cotas dominantes verificadas son:

Tabla 1: Complejidad temporal y espacial por componente.

Componente	Temporal	Espacial	Cuello de botella
marginalize	$\mathcal{O}(2^m)$	$\mathcal{O}(2^m)$	np.mean (L2)
CostTable (build)	$\mathcal{O}(m2^m)$	$\mathcal{O}(2^m)$	BFS por niveles
cut pool (KGeoMIP)	$\mathcal{O}(n)$ cortes	$\mathcal{O}(n)$	—
δ_k (una candidata)	$\mathcal{O}(2^n)$	$\mathcal{O}(2^n)$	marginalización
GREEDY (k bloques)	$\mathcal{O}(k^2 n 2^n)$	$\mathcal{O}(2^n)$	evaluaciones δ_k
QNodes (Queyranne)	$\mathcal{O}(n^3 2^n)$	$\mathcal{O}(2^n)$	marginalización
Exacto (Stirling)	$\mathcal{O}(S(2n, k) 2^n)$	$\mathcal{O}(2^n)$	enumeración

Temporal. KGeoMIP $\approx \mathcal{O}(m2^m + k^2 n 2^n)$; KQNodes $\approx \mathcal{O}(n^3 2^n)$ (la búsqueda submodular domina). Frente a la fuerza bruta $\mathcal{O}(S(2n, k) 2^n)$, ambas heurísticas reducen el factor combinatorio $S(2n, k)$ a uno polinómico en n y k .

Espacial. El término dominante es el tensor del subsistema y las distribuciones, $\mathcal{O}(2^n)$. En float32 esto es la mitad que en float64: para $n = 25$, $\approx 3,35$ GB en lugar de 6,7 GB, lo que hace al sistema cargable en RAM.

Análisis de casos. El **mejor caso** ocurre cuando el primer corte ya aísla la masa de probabilidad (pérdida casi nula, p. ej. N15A con $\delta_k = 0,027$). El **peor caso** aparece cuando ningún corte del pool reduce δ_k y deben probarse todos los pares (bloque, corte) en los $k - 1$ niveles; el costo crece con k y con la densidad de la TPM. La marginalización $\mathcal{O}(2^m)$ es el cuello empírico: 87 ms a $m=20$ y 1,45 s a $m=24$.

6. Detalles de implementación

6.1. Métodos principales

- KGeoMIP(tpm, initial_state, k) y KQNodes(tpm, initial_state, k) — construyen la estrategia para un k dado.
- apply_strategy(condition, purview, mechanism) -> Solution — prepara el subsistema, construye el *cut pool* y delega en greedy_k_partition; devuelve un Solution (estrategia, pérdida, partición, tiempo).
- System.k_partition(partition) -> System — aplica la reconstrucción factorizada de una KPartition validada.
- delta_k(subsystem, partition, baseline) -> (float, ndarray) — pérdida δ_k y distribución particionada.
- KPartition.from_blocks(blocks, future_universe, present_universe) — fábrica que valida y normaliza los bloques.

```

1 def delta_k(subsystem, partition, baseline_distribution=None):
2     original = baseline_distribution
3     if original is None:
4         original = subsystem.marginal_distribution()
5     partitioned = subsystem.k_partition(partition).marginal_distribution()
6     return effect_emd(original, partitioned), partitioned

```

Listing 1: Núcleo de la evaluación δ_k (src/funcs/emd.py).

6.2. Dependencias externas

Tabla 2: Dependencias principales y su uso.

Biblioteca	Versión	Uso
numpy	2.4.6	tensores, vectorización, EMD analítica
scipy	1.17.1	clustering espectral (<code>csgraph</code> , <code>linalg</code>)
pandas	3.0.3	E/S de TPM y rejilla de resultados
more_itertools	—	particiones de Stirling (exacto)
matplotlib	—	figuras y visualización de particiones
pyphi	—	validación cruzada (6–10 nodos)
joblib / psutil	—	PCD (procesos), medición
numba (opc.)	0.65.1	kernel EMD por lotes (umbral)
pyemd (opc.)	2.0.0	EMD causal (métrica Hamming) de comprobación
plotly (opc.)	6.8.0	figuras interactivas (extra viz)
streamlit (opc.)	1.58.0	interfaz web (extra web)

6.3. Ingeniería de software

- **Validación de entradas:** `KPartition` verifica cobertura, disyunción y no-vacuidad estricta; `apply_strategy` exige $k \geq 2$.
- **Manejo de errores:** `ValueError` (parámetros/partición inválida) y `RuntimeError` (pool incapaz de refinar a k bloques) con mensajes descriptivos.
- **Logging:** `SafeLogger` (carpeta logs/); profiling con `@profile` (HTML en review/profiling/).
- **Reproducibilidad:** semilla central en `application`; semillas derivadas por proceso con `SeedSequence.spawn`.

6.4. Tests implementados

La batería (`uv run pytest`, 206 casos) cubre: regresión $k = 2$ ($\text{KGeoMIP} \equiv \text{GeoMIP}$, $\text{KQNodes} \equiv \text{QNodes}$), validación $k \in \{3, 4, 5\}$ contra el exacto, semántica estricta de `KPartition`, determinismo del clustering y de las **metaheurísticas** (misma semilla $\Rightarrow$ misma partición), cota $\delta_k \geq \delta_k^*$ de las siete estrategias, integración *end-to-end* (cargar $\rightarrow$ estrategia $\rightarrow$ `Solution` y *roundtrip* generar/cargar), EMD causal opcional (`importorskip`), figuras interactivas, registro headless (`runner`), precisión `float32` vs `float64` ($|\Delta| \leq 10^{-5}$), métricas y PCD. Triage histórico de `QNodes` congelado en `tests/unit/test_qnodes_triage.py`.

7. Resultados experimentales

7.1. Datasets

TPMs deterministas `N{n}{letra}.csv` (2^n filas $\times n$ columnas, little-endian), generadas con semilla 73. La rejilla de evaluación sigue una estrategia **híbrida honesta**: $n \in \{10, 15\}$ con las *siete* estrategias (KGeoMIP , KQNodes , `Clustering  $\times 2$` , `GA/SA/Tabú`) y δ_k real; $n = 20$ solo con KGeoMIP (las demás del núcleo se vuelven impracticables, ver abajo); y $n = 25$ solo con clustering (que *propone* la partición pero deja δ_k en blanco por memoria, §8). En todos los casos $k \in \{2, 3, 4\}$. El consolidado reproducible vive en `benchmark_results_FINAL.csv`.

7.2. Métricas

- **Pérdida δ_k** (Ec. (1)): menor es mejor.
- **Tiempo de ejecución** (s, perfilador apagado).
- **Tasa de acierto exacto**: fracción de casos donde δ_k iguala al óptimo exacto (n pequeño).
- **Error relativo de Φ y distancia de Jaccard** entre particiones (conteo de pares co-asignados).
- **Speedup y pendiente de escalabilidad** (log–log).

7.3. Tabla de resultados

Tabla 3: Pérdida δ_k y tiempo por estrategia (rejilla FINAL).

Red	k	KGeoMIP δ_k	KQNodes δ_k	Clustering δ_k (mejor)
N10A	2	0,4697	0,4697	4,3438
N10A	3	0,9424	0,9424	4,8594
N10A	4	1,4229	1,4229	4,8594
N15A	2	0,0268	0,0268	0,5274
N15A	3	0,0538	0,0539	0,8752
N15A	4	0,0809	0,0810	1,0762
N20A	2	0,4993	—	—
N20A	3	0,9987	—	—
N20A	4	1,4982	—	—

En N20A solo KGeoMIP es practicable (~ 82 s/k); «—» marca estrategias no ejecutadas por inviabilidad temporal (KQNodes) o de memoria (clustering sin δ_k).

Tabla 4: Metaheurísticas vs óptimo (KGeoMIP = δ_k^* en esta rejilla).

Red	k	Óptimo	Tabú	Genética	Recocido
N10A	2	0,4697	0,4697	0,4697	0,4727
N10A	3	0,9424	0,9424	2,3809	0,9639
N10A	4	1,4229	1,4229	3,8848	1,4365
N15A	2	0,0268	0,0268	0,0268	0,0271
N15A	3	0,0538	0,0538	0,5561	0,0540
N15A	4	0,0809	0,0809	0,9198	0,8346

Tabla 5: Tiempo de ejecución (s), perfilador apagado.

Red	k	KGeoMIP (s)	KQNodes (s)
N10A	2	0,051	0,181
N10A	3	0,048	0,193
N10A	4	0,055	0,176
N15A	2	1,619	4,924
N15A	3	1,651	4,487
N15A	4	1,544	4,494
N20A	2	83,77	—
N20A	3	83,31	—
N20A	4	80,94	—

7.4. Gráficas y visualizaciones

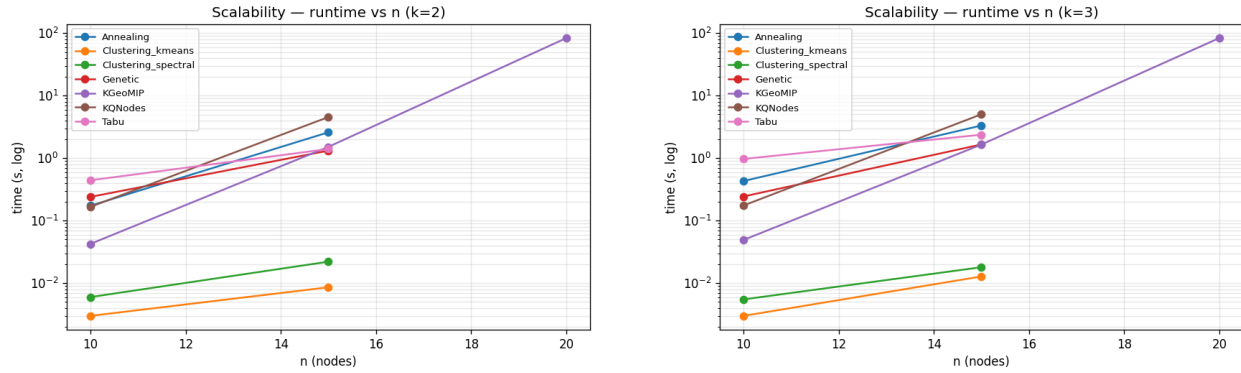


Figura 5: Escalabilidad (tiempo vs n , escala log) para $k=2$ (izq.) y $k=3$ (der.). KGeoMIP es consistentemente más rápido que KQNodes y es la única estrategia núcleo que alcanza $n=20$ (~ 82 s).

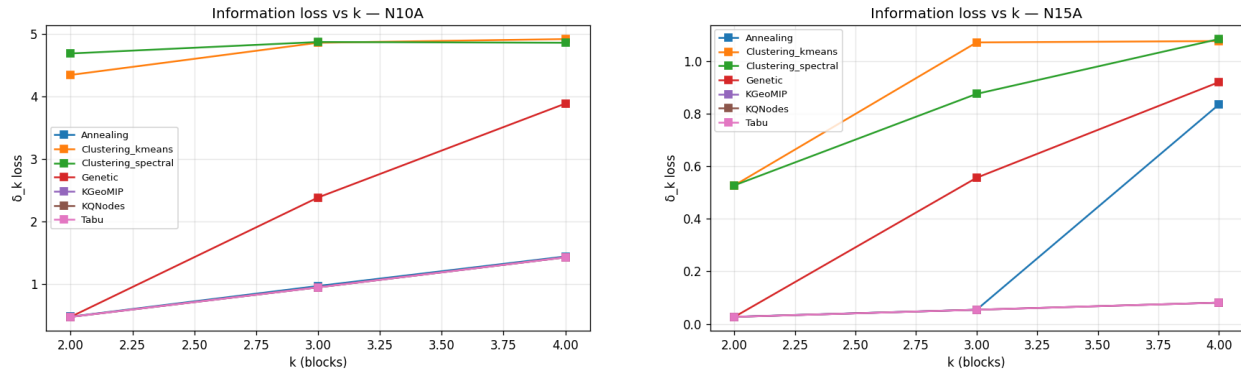


Figura 6: Pérdida δ_k vs k en N10A (izq.) y N15A (der.): monótona creciente, como exige la semántica estricta.

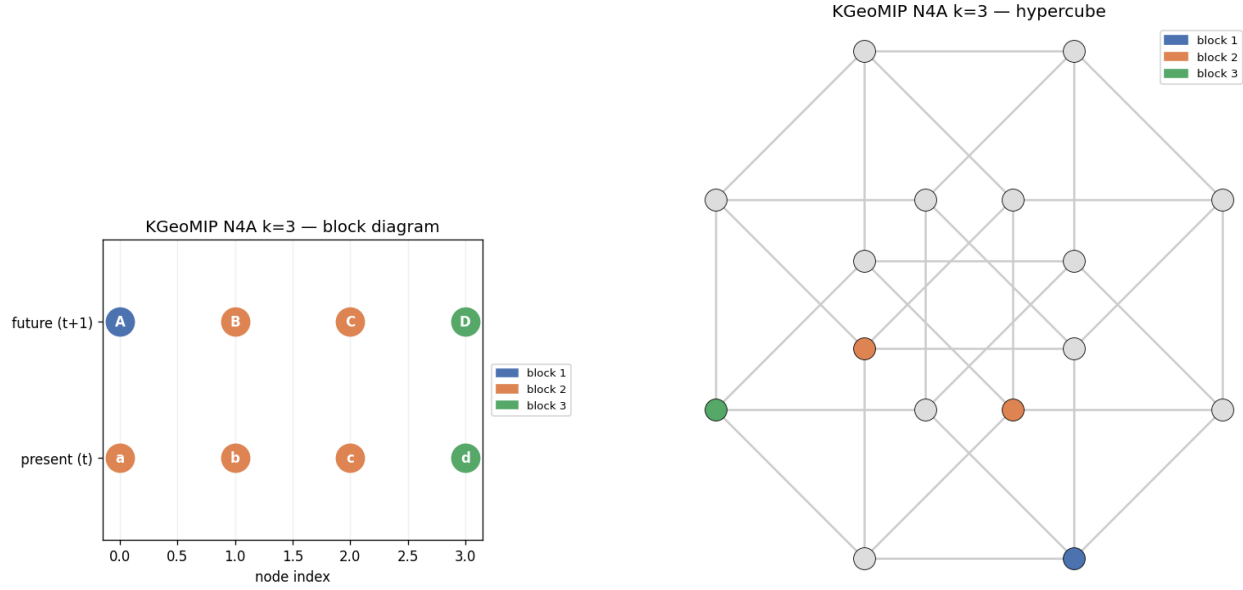


Figura 7: k -partición hallada (N4A, $k = 3$): diagrama de bloques presente/futuro (izq.) y proyección del hipercubo de nodos (der.).

7.5. Visualización interactiva e interfaz web

Las mismas vistas se ofrecen de forma **interactiva** (Plotly) mediante `src/viz/interactive.py`: diagrama de bloques de la partición con *hover* por bloque, δ_k vs k y escalabilidad con ejes navegables. `scripts/make_interactive.py` las exporta como HTML autocontenido (plotly.js por CDN). Todo se integra en una **interfaz web** (Streamlit, `app/streamlit_app.py`) que permite, sin línea de comandos: generar una TPM desde el navegador, elegir estrategia/ k /máscaras, y obtener la partición, su δ_k , la distribución marginal y las figuras estática e interactiva. El registro headless `src/funcs/runner.py` (`run_analysis`) es el núcleo compartido por la UI y los scripts.

7.6. Análisis de resultados

- **KGeoMIP $\equiv$ KQNodes en calidad:** ambas hallan la misma pérdida (diferencias $\leq 10^{-4}$ por desempates), confirmando que comparten el motor voraz y la métrica δ_k .
- **Núcleo $\gg$ baseline:** las estrategias núcleo superan al clustering por $\sim 10\times$ en N10A y $\sim 20\times$ en N15A; el clustering optimiza una métrica de grafo, no δ_k .
- **Costo de KQNodes:** $\sim 3\times$ más lento que KGeoMIP por la búsqueda Queyranne $\mathcal{O}(n^3)$, sin ganancia de calidad en esta rejilla.
- **Monotonía:** δ_k crece con k (Fig. 6), coherente con que más bloques destruyen más información.
- **KGeoMIP escala a $n = 20$:** computa δ_k en ~ 82 s/ k (Tabla 5), un punto empírico que valida la cota $\mathcal{O}(2^n)$ (de $n = 15$ a $n = 20$, $2^5 = 32\times$ más estados $\Rightarrow$ salto de $\sim 1,6$ s a ~ 82 s, factor $\sim 50\times$, dominado por la marginalización). KQNodes no termina en tiempo de sesión ahí (Queyranne $\mathcal{O}(n^3 2^n)$).
- **Metaheurísticas (Tabla 4):** la *búsqueda tabú* iguala el óptimo en toda la rejilla; el *recocido* se le aproxima salvo en k alto; la *genética* acierta en $k = 2$ pero se degrada al crecer k , pues el espacio k^{2n} excede el presupuesto de ~ 1200 evaluaciones por defecto. Es el comportamiento esperado de una

metaheurística sin reinicio y refuerza que las estrategias núcleo (deterministas) son la mejor opción aquí.

7.7. Validación de correctitud

- **Regresión $k = 2$:** KGeoMIP y KQNodes reproducen exactamente GeoMIP y QNodes legados.
- **Oráculo:** sobre los casos golden, el núcleo ($\text{BruteForce} \equiv \text{GeoMIP}$) coincide 21/21; KGeoMIP iguala el óptimo exacto y KQNodes acierta 9/10 (corrige el defecto N3B de QNodes: 0,46875 vs 0,5).
- **Consistencia:** $\text{loss} = \delta_k$ y $\text{loss} \geq$ óptimo exacto en 18/18 casos del verificador; float32 igual a float64 dentro de 10^{-5} .

8. Limitaciones y trabajo futuro

8.1. Limitaciones conocidas

- **Techo de escala:** KGeoMIP/KQNodes son prácticos hasta $n \approx 15\text{--}20$ (marginalización $\mathcal{O}(2^n)$, Queyranne $\mathcal{O}(n^3)$). A $n = 20$, $\text{KGeoMIP}_{k=2} \approx 52\text{--}84$ s y $\text{KQNodes} \approx 348$ s.
- $n = 25$: solo el clustering produce una partición (en < 5 s), pero *no* se puntúa con δ_k porque requiere el System completo en memoria (OOM). Las filas N25A reportan partición sin pérdida, documentado honestamente.
- **Optimalidad:** el motor voraz es heurístico; no garantiza la k -MIP global (sí factibilidad, monotonía y validación contra el exacto en n pequeño).

8.2. Supuestos y restricciones

TPM determinista binaria; independencia condicional del repertorio de efecto (que habilita la EMD-efecto analítica $\mathcal{O}(n)$); GIL activo en Python 3.14.5, por lo que el paralelismo es por procesos, no por hilos.

8.3. Mejoras potenciales y trabajo futuro

- Marginalización *local* $\mathcal{O}(2^{\text{dim}})$ por bloque para subir el techo de n .
- Refinamiento con *look-ahead* o *beam search* para acercarse al óptimo global manteniendo costo polinómico.
- Baseline de metaheurísticas (GA/SA/Tabú), diferido (Fase 5B).
- Aceleración GPU solo si el volumen justifica la transferencia H2D (analizado: no rentable para los tamaños actuales).

9. Apéndices técnicos

9.1. Reducción $k = 2 \Rightarrow$ bipartición legada

Proposición. Para $k = 2$, GREEDYKPARTITION ejecuta exactamente un refinamiento y el resultado coincide con la bipartición de la estrategia base.

Bosquejo. El bucle while corre mientras $|\text{blocks}| < 2$, es decir una sola iteración partiendo de $[(F, M)]$. El

único bloque es el universo completo, de modo que dividirlo con un corte c produce $(c, \bar{c})$, que es la bipartición candidata original. Como el pool y la métrica δ_k son los del método base, se selecciona la misma bipartición. $\square$

9.2. Reproducción de figuras y diagramas

```

1 # Rejilla y métricas
2 uv run scripts/run_benchmark.py
3 uv run scripts/validate_correctness.py
4 # Figuras (matplotlib) y visualización de particiones
5 uv run scripts/make_figures.py
6 uv run scripts/make_viz.py --net N4A --k 3 --strategy KGeoMIP
7 # Diagramas UML editables (PlantUML)
8 plantuml docs/manuales/diagrams/*.puml

```

Listing 2: Comandos de reproducción.

9.3. Referencias

Referencias

- [1] G. Tononi. *An information integration theory of consciousness*. BMC Neuroscience, 2004.
- [2] W. Mayner et al. *PyPhi: A toolbox for integrated information theory*. PLoS Comput. Biol., 2018.
- [3] M. Queyranne. *Minimizing symmetric submodular functions*. Mathematical Programming, 1998.
- [4] Universidad de Caldas. *Proyecto K-QGMIP — Especificación* (docs/Proyecto_KQMIP.md), 2026-1.

9.4. Uso de inteligencia artificial generativa

Durante el desarrollo se utilizó **Claude Code (Opus 4.8)** como asistente, documentado entrada por entrada en logs/ai_agent_changelog.md (con el *prompt* del usuario, parámetros reales y decisiones).

Etapas y ejemplos.

- **Diseño:** extracción del motor voraz compartido (greedy_k_partition) y decisión de semántica estricta tras medir que la semántica débil degeneraba $k > 2$ a bipartición.
- **Implementación:** vectorización de CostTable, conversión a float32, infraestructura PCD (parallel.py) y visualización (src/viz).
- **Depuración/optimización:** perfilado con pyinstrument; hallazgo de que Numba en lotes pequeños era $\sim 70\%$ más lento (revertido) y de que la marginalización domina el costo.
- **Documentación:** estructuración de manuales, diagramas y tablas.

Reflexión crítica. La IA aceleró tareas mecánicas (vectorización, *boilerplate*, redacción) y propuso refactorizaciones útiles; sin embargo, varias afirmaciones de evaluación automática se basaron en código obsoleto y debieron verificarse contra el repositorio real (regla “verificar, no asumir”). El criterio de validación contra oráculo y *microbenchmarks* aislados fue imprescindible para no aceptar mejoras ilusorias. La comprensión profunda del algoritmo y la responsabilidad de las decisiones permanecen en el equipo.